11.03 Entity–relationship modelling

We can use a top-down method called stepwise refinement to break down the process of database design into simple steps (see also Chapter 12, Section 12.08). At each step more detail is added to the design. In database design this approach uses an entity-relationship (E–R) diagram. Typically, this can be created either by a database designer or a systems analyst working with the designer. We introduced the term 'relationship' earlier in connection with the use of a foreign key. An entity (strictly speaking an entity type) could be a thing, a type of person, an event, a transaction or an organisation. Most importantly, there must be a number of 'instances' of the entity. An entity is something that will become a table in a relational database.

WORKED EXAMPLE 11.01

Creating an entity–relationship diagram for the theatrical agency

Let's consider a scenario for the theatrical agency which will be sufficient to model a part of the final database they would need. The starting point for a top-down design is a statement of the requirement:

The agency needs a database to handle bookings for bands. Each band has a number of members. Each booking is for a venue. Each booking might be for one or more bands.

Step 1: Choose the entities

You look for the nouns. You ignore 'agency' because there is only the one. You choose Booking, Band, Member and Venue. For each of these there will be more than one instance. You are aware that each booking is for a gig at a venue but you ignore this because you think that the Booking entity will be sufficient to hold the required data about a gig.

Step 2: Identify the relationships

This requires experience, but the aim is not to define too many. You choose the following three:

- Booking with Venue
- Booking with Band
- Band with Member.

You ignore the fact that there will be, for example, a relationship between Member and Venue because you think that this will be handled through the other relationships that indirectly link them. You can now draw a preliminary E–R diagram as shown in Figure 11.02.



Figure 11.02 A preliminary entity–relationship diagram

Step 3: Decide the cardinalities of the relationships

Now comes the crucial stage of deciding on what are known as the 'cardinalities' of the relationships. At present we have a single line connecting each pair of entities. This line actually defines two relationships which might be described as the 'forward' one and the 'backward' one on the diagram as drawn. However, this only becomes apparent at the final stage of drawing the relationship. First, we have to choose one of the following descriptions for the cardinality of each relation:

- one-to-one or 1:1
- one-to-many or 1:M
- many-to-one or M:1
- many-to-many or M:M.

(Continued)

Let's consider the relationship between Member and Band. We argue that one Member is a member of only one Band. (This needs to be confirmed as a fact by the agency.) We then argue that one Band has more than one Member so it has many. Therefore, the relationship between Member and Band is M:1. In its simplest form, this relationship can be drawn as shown in Figure 11.03.



Figure 11.03 The M:1 relationship between Member and Band

This can be given more detail by including the fact that a member must belong to a Band and a Band must have more than one Member. To reflect this, the relationship can be drawn as shown in Figure 11.04.



Figure 11.04 The M:1 relationship with more detail

At each end of the relationship there are two symbols. One of the symbols shows the minimum cardinality and the other the maximum cardinality. In this particular case, the minimum and maximum values just happen to be the same. However, using the diagram to document that a Member must belong to a Band is important. It indicates that when the database is created it must not be possible to create a new entry in the Member table unless there is a valid entry for BandName in that table.

For the relationship between Booking and Venue we argue that one Booking is for one Venue (there must be a venue and there cannot be more than one) and that one Venue can be used for many Bookings so the relationship between Booking and Venue is M:1. However, a Venue might exist that has so far never had a booking so the relationship can be drawn as shown in Figure 11.05.



Figure 11.05 The M:1 relationship between Booking and Venue

Finally for the relationship between Band and Booking we argue that one Booking can be for many Bands and that one Band has many Bookings (hopefully!) so the relationship is M:M. However, a new band might not yet have a booking. Also, there might be only one Band for a booking so the relationship can be drawn as shown in Figure 11.06.



Figure 11.06 The M:M relationship between Band and Booking

Step 4: Create the full E–R diagram

At this stage we should name each relationship. The full E–R diagram for the limited scenario that has been considered is as shown in Figure 11.07.

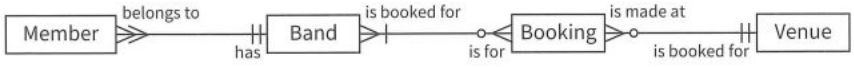


Figure 11.07 The E–R diagram for the theatrical agency's booking database

To illustrate how the information should be read from such a diagram we can look at the part shown in Figure 11.08. Despite the fact that there is a many-to-many relationship, a reading of a relationship always considers just one entity to begin the sentence. So, reading forwards and then backwards, we say that:

(Continued)

One Band is booked for zero or many Bookings

One Booking is for one or many Bands

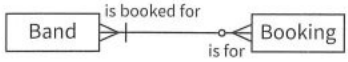


Figure 11.08 Part of the annotated E–R diagram

Question 11.02

If you are deciding on the cardinality of the relationship between two entities does it matter which one is put on the left and which on the right?

TIP

Be careful not to confuse the two completely different terms relation and relationship.

11.04 A logical entity–relationship model

A fully annotated E–R diagram of the type developed in Section 11.03 holds all of the information about the relationships that exist for the data that is to be stored in a system. It can be defined as a conceptual model because it does not relate to any specific way of implementing a system. If the system is to be implemented as a relational database, the E–R diagram has to be converted to a logical model. To do this we can start with a simplified E–R diagram that just identifies cardinalities.

If a relationship is 1:M, no further refinement is needed. The relationship shows that the entity at the many end needs to have a foreign key referencing the primary key of the entity at the one end.

If there were a 1:1 relationship there are options for implementation. However, such relationships are extremely rare and we do not need to consider them here.

The problem relationship is the M:M, where a foreign key cannot be used. A foreign key attribute can only have a single value, so it cannot handle the many references required. Another way of looking at this problem is to argue that a foreign key is required in each entity but neither table could be created first because the other table needed to exist for the foreign key to be defined. The solution for the M:M relationship is to create a link entity. For Band and Booking, the logical entity model will contain the link entity shown in Figure 11.09.

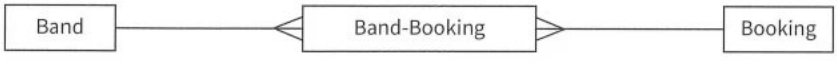


Figure 11.09 A link entity inserted to resolve a M:M relationship

Extension Question 11.01

Is it possible to annotate these relationships?

With the link entity in the model it is now possible to have two foreign keys in the link entity; one referencing the primary key of Band and one referencing the primary key of Booking.

Each entity in the logical E–R diagram will become a table in the relational database. It is therefore possible to choose primary keys and foreign keys for the tables. These can be summarised in a key table. Table 11.04 shows sensible choices for the theatrical agency's booking database.

Table name	Primary key	Foreign key
Member	MemberID	BandName
Band	BandName	
Band-Booking	BandName & BookingID	BandName, BookingID
Booking	BookingID	VenueName
Venue	VenueName	

Table 11.04 A key table for the agency booking database

The decisions about the primary keys are determined by the uniqueness requirement. The link entity cannot use either BandName or BookingID alone but the combination of the two in a compound primary key will work.

TASK 11.01

Consider the following scenario. An organisation books cruises for passengers. Each cruise visits a number of ports. Create a conceptual E–R diagram and convert it to a logical E–R diagram. Create a key table for the database that could be implemented from the design.

11.05 Normalisation

Normalisation is a design technique for constructing a set of table designs from a list of data items. It can also be used to improve on existing table designs.



TIP

Unfortunately, you will be coming across a completely different use of the term normalisation in Chapter 16.

WORKED EXAMPLE 11.02

Normalising data for the theatrical agency

To illustrate the technique, let's consider the document shown in Figure 11.10. This is a booking data sheet that the theatrical company might use.

Booking data sheet: 2016/023
Venue:
Cambridge International Theatre
Camside
CA1
Booking date: 23.06.2016

Bands booked	Number of band members	Headlining
ComputerKidz	5	Y
ITWizz	3	N
DeadlyDuo	2	N

Figure 11.10 Example booking data sheet

(Continued)

The data items on this sheet (ignoring headings) can be listed as a set of attributes:

(BookingID, VenueName, VenueAddress1, VenueAddress2, Date,

(BandName, NumberOfMembers, Headlining))

The list is put inside brackets because we are starting a process of table design. The extra set of brackets around BandName, NumberOfMembers, Headlining is because they represent a **repeating group**. If there is a repeating group, the attributes cannot sensibly be put into one relational table. A table must have single rows and atomic attribute values so the only possibility would be to include tuples such as those shown in Table 11.05. There is now data redundancy here with the duplication of the BookingID, venue data and the date.

Booking ID	Venue Name	Venue Address1	Venue Address2	Date	Band Name	Number Of Members	Headlining
2016/023	Cambridge International Theatre	Camside	CA1	23.06.2016	Computer Kidz	5	Y
2016/023	Cambridge International Theatre	Camside	CA1	23.06.2016	ITWizz	3	N
2016/023	Cambridge International Theatre	Camside	CA1	23.06.2016	DeadlyDuo	2	N

Table 11.05 Data stored in an unnormalised table

Step 1: Conversion to first normal form (1NF)

The conversion to first normal form (1NF) requires splitting the data into two groups. At this stage we represent the data as table definitions. Therefore, we have to choose table names and identify a primary key for each table. One table contains the non-repeating group attributes, the other the repeating group attributes. For the first table a sensible design is:

Booking(BookingID, VenueName, VenueAddress1, VenueAddress2, Date)

The table with the repeating group is not so straightforward. It needs a compound primary key and a foreign key to give a reference to the first table. The sensible design is:

Band-Booking(BandName, BookingID(fk), NumberOfMembers, Headlining)

Again, the primary key is underlined but also the foreign key has been identified, with (fk). Because the repeating groups have been moved to a second table, these two tables could be implemented with no data redundancy in either. This is one aspect of 1NF. Also, we can say that for each table the attributes are dependent on the primary key.

Step 2: Conversion to second normal form (2NF)

For conversion to second normal form (2NF), the process is to examine each non-key attribute and ask if it is dependent on both parts of the compound key. Any attributes that are dependent on only one of the attributes in the compound key must be moved out into a new table. In this case, NumberOfMembers is only dependent on BandName. In 2NF there are now three table definitions:

Booking(BookingID, VenueName, VenueAddress1, VenueAddress2, Date)

Band-Booking(BandName(fk), BookingID(fk), Headlining)

Band(BandName, NumberOfMembers)

(Continued)